

HDOC DAY - 18

- **Strong Number:**

A number whose sum of the factorials of its digits equals the original number.

Example: $145 = 1! + 4! + 5! = 145$

- **Armstrong Number:**

A number equal to the sum of each digit raised to the power of the total number of digits.

Example: $153 = 1^3 + 5^3 + 3^3 = 153$

- **Harshad Number:**

A number divisible by the sum of its digits.

Example: $18 \rightarrow 1 + 8 = 9$, and $18 \% 9 = 0$

- **Duck Number:**

A positive number that contains at least one 0 digit.

Example: 1023 is a Duck number.

Algorithm:

Main Program

1. Start.
2. Display the menu:
 - Check Strong Number
 - Check Armstrong Number
 - Check Harshad Number
 - Check Duck Number
 - Exit

3. Input the user's choice.
4. Use switch(choice).
5. For choice 1, call strongNumber().
6. For choice 2, call armstrongNumber().
7. For choice 3, call harshadNumber().
8. For choice 4, call duckNumber().
9. For choice 5, terminate the program.
10. For any other choice, display "Invalid choice".
11. Stop.

Algorithm for Strong Number:

1. Input a positive integer n.
2. Store n in original.
3. Initialize sum = 0.
4. Extract each digit using % 10.
5. Find the factorial of each digit.
6. Add the factorial to sum.
7. Remove the last digit using / 10.
8. Repeat until all digits are processed.
9. If sum == original, it is a Strong number.
10. Otherwise, it is not a Strong number.

Algorithm for Armstrong Number:

1. Input a positive integer n.
2. Store n in original.
3. Count the number of digits.
4. Extract each digit.

5. Calculate digit raised to the power count.
6. Add the result to sum.
7. Repeat for all digits.
8. If $\text{sum} == \text{original}$, it is an Armstrong number.
9. Otherwise, it is not an Armstrong number.

Algorithm for Harshad Number:

1. Input a positive integer n.
2. Store n in original.
3. Initialize $\text{sum} = 0$.
4. Extract each digit.
5. Add each digit to sum.
6. Repeat until all digits are processed.
7. Check $\text{original} \% \text{sum}$.
8. If the remainder is 0, it is a Harshad number.
9. Otherwise, it is not a Harshad number.

Algorithm for Duck Number:

1. Input a positive integer n.
2. Initialize $\text{flag} = 0$.
3. Extract each digit using $\% 10$.
4. If a digit is 0, set $\text{flag} = 1$.
5. Repeat until all digits are checked.
6. If $\text{flag} == 1$, it is a Duck number.
7. Otherwise, it is not a Duck number.

C program:

```
#include <stdio.h>
```

```
/* Function declarations */
void strongNumber(void);
void armstrongNumber(void);
void harshadNumber(void);
void duckNumber(void);

int main()
{
    int choice;

    printf("\n--- DIGIT BASED OPERATIONS ---\n");
    printf("1. Check Strong Number\n");
    printf("2. Check Armstrong Number\n");
    printf("3. Check Harshad Number\n");
    printf("4. Check Duck Number\n");
    printf("5. Exit\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            strongNumber();
```

```
break;
```

```
case 2:
```

```
    armstrongNumber();
```

```
    break;
```

```
case 3:
```

```
    harshadNumber();
```

```
    break;
```

```
case 4:
```

```
    duckNumber();
```

```
    break;
```

```
case 5:
```

```
    printf("Program terminated.\n");
```

```
    break;
```

```
default:
```

```
    printf("Invalid choice.\n");
```

```
}
```

```
return 0;
```

```
}
```

```
/* Function to check Strong Number */
```

```
void strongNumber(void)
```

```
{
```

```
    int n, original, digit;
```

```
    int sum = 0;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d", &n);
```

```
    original = n;
```

```
    while (n > 0)
```

```
    {
```

```
        int factorial = 1;
```

```
        int i;
```

```
        digit = n % 10;
```

```
        for (i = 1; i <= digit; i++)
```

```
        {
```

```
            factorial = factorial * i;
```

```
        }
```

```
        sum = sum + factorial;
```

```
    n = n / 10;
```

```
}
```

```
if (sum == original)
```

```
    printf("%d is a Strong number.\n", original);
```

```
else
```

```
    printf("%d is not a Strong number.\n", original);
```

```
}
```

```
/* Function to check Armstrong Number */
```

```
void armstrongNumber(void)
```

```
{
```

```
    int n, original, temp, digit;
```

```
    int count = 0, sum = 0;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d", &n);
```

```
    original = n;
```

```
    temp = n;
```

```
/* Count the number of digits */
```

```
while (temp > 0)
```

```
{
```

```
    count++;  
    temp = temp / 10;  
}  
  
temp = n;  
  
/* Calculate Armstrong sum */  
while (temp > 0)  
{  
    int power = 1;  
    int i;  
  
    digit = temp % 10;  
  
    for (i = 1; i <= count; i++)  
    {  
        power = power * digit;  
    }  
  
    sum = sum + power;  
    temp = temp / 10;  
}  
  
if (sum == original)  
    printf("%d is an Armstrong number.\n", original);
```

```
else  
    printf("%d is not an Armstrong number.\n", original);  
}
```

```
/* Function to check Harshad Number */
```

```
void harshadNumber(void)
```

```
{
```

```
    int n, original, digit;
```

```
    int sum = 0;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d", &n);
```

```
    original = n;
```

```
    while (n > 0)
```

```
    {
```

```
        digit = n % 10;
```

```
        sum = sum + digit;
```

```
        n = n / 10;
```

```
    }
```

```
    if (original % sum == 0)
```

```
        printf("%d is a Harshad number.\n", original);
```

```
else  
    printf("%d is not a Harshad number.\n", original);  
}
```

```
/* Function to check Duck Number */
```

```
void duckNumber(void)
```

```
{
```

```
    int n, original, digit;
```

```
    int flag = 0;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d", &n);
```

```
    original = n;
```

```
    while (n > 0)
```

```
    {
```

```
        digit = n % 10;
```

```
        if (digit == 0)
```

```
        {
```

```
            flag = 1;
```

```
            break;
```

```
        }
```

```
n = n / 10;  
}
```

```
if (flag == 1)  
    printf("%d is a Duck number.\n", original);  
else  
    printf("%d is not a Duck number.\n", original);  
}
```

Compilation and execution:

```
(kali㉿kali)-[~]  
$ vi h18.c  
  
(kali㉿kali)-[~]  
$ gcc h18.c -o h18  
  
(kali㉿kali)-[~]  
$ ./h18
```

Test case table:

Test case	Choice	Input	Expected output
1	1	145	145 is a strong number
2	1	123	123 is not a strong number
3	2	153	153 is an Armstrong number
4	2	123	123 is not an Armstrong number
5	3	18	18 is a harshad number

6	3	19	19 is not a harshad number
7	4	1023	1023 is a duck number
8	4	1234	1234 is not a duck number
9	5	--	Program terminated
10	8	--	Invalid choice

```

— DIGIT BASED OPERATIONS —
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
5. Exit
Enter your choice: 1
Enter a positive integer: 145
145 is a Strong number.

```

```

(kali㉿kali)-[~]
$ ./h18

```

```

— DIGIT BASED OPERATIONS —
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
5. Exit
Enter your choice: 1
Enter a positive integer: 123
123 is not a Strong number.

```

```

(kali㉿kali)-[~]
$ ./h18

```

```

— DIGIT BASED OPERATIONS —
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
5. Exit
Enter your choice: 2
Enter a positive integer: 153
153 is an Armstrong number.

```

```

(kali㉿kali)-[~]
$ ./h18

```